Artificial neural networks. An artificial neuron computes a weighted sum of its inputs, adds a bias, and applies a non-linear activation function. A single neuron with a step activation is the perceptron, which can only represent linearly separable functions. Stacking neurons into layers and adding at least one hidden layer lets the network represent non-linear functions. The universal approximation theorem states that a network with a single hidden layer and enough neurons can approximate any continuous function on a compact domain, although it does not say how many neurons are needed or how to find the weights. Common activation functions include the sigmoid, the hyperbolic tangent, and the rectified linear unit, or ReLU, which outputs zero for negative inputs and the input itself otherwise. ReLU is cheap to compute and reduces the vanishing gradient problem that affects sigmoid and tanh in deep networks.

Training with backpropagation. Neural networks are trained by minimizing a loss function over the training data. Backpropagation is the algorithm that computes the gradient of the loss with respect to every weight. It applies the chain rule layer by layer, starting from the output and moving backwards through the network, reusing intermediate results at each step. In practice the gradients are computed on a mini-batch, which is a small random subset of the training data, and the weights are updated with stochastic gradient descent. Momentum accumulates a running average of past gradients to smooth the updates. Adam combines momentum with a per-parameter scaling based on recent squared gradients, and it is a common default because it often works well with little tuning. An epoch is one full pass over the training set.